

第5节-概率密度函数与函数变换

五、概率密度函数与函数变换

5.1 概率密度 vs 累积分布函数

5.1.1 基本定义与直观理解

随机变量：设 X 是一个连续随机变量，它的取值是实数集合上的连续区间。

累积分布函数（CDF）

- 定义： $F_X(x)$ 表示随机变量 X 取值小于或等于 x 的概率。

$$F_X(x) = P(X \leq x)$$

- 取值范围： $F_X(x)$ 单调非减，且满足：

$$\lim_{x \rightarrow -\infty} F_X(x) = 0, \quad \lim_{x \rightarrow +\infty} F_X(x) = 1$$

概率密度函数（PDF）： $f_X(x)$ 是随机变量 X 在点 x 的概率“密度”。

对于连续随机变量，若其累积分布函数， $F_X(x)$ 在 x 处可导，则其概率密度函数为：

$$f_X(x) = \frac{d}{dx} F_X(x)$$

此时 $P(X = x) = 0$ 。

5.1.2 从概率密度函数到概率计算

单点概率，对于连续随机变量：

$$P(X = x) = 0$$

因为连续变量的取值是无限多个点，单点概率趋近于0。

区间概率

概率密度函数的实用意义体现在计算区间概率：

$$P(a \leq X \leq b) = \int_a^b f_X(x) dx = F_X(b) - F_X(a)$$

- 积分曲线下的面积对应概率

$F_X(x)$ 就是从 $-\infty$ 积分到 x 的面积

形象解释

- **CDF** 是从左到 x 的累计概率，像是“累计水量”；
- **PDF** 的单位是“概率/单位长度”，是每个点处的“水流密度”，曲线下的面积。

5.1.3 具体性质及公式推导

从 CDF 求 PDF，假设 $F_X(x)$ 可导，则：

$$f_X(x) = \frac{dF_X(x)}{dx}$$

证明思路（微积分基本定义）：

$$f_X(x) = \lim_{\Delta x \rightarrow 0} \frac{F_X(x + \Delta x) - F_X(x)}{\Delta x} = \frac{dF_X(x)}{dx}$$

从 PDF 求 CDF

由积分定义：

$$F_X(x) = \int_{-\infty}^x f_X(t) dt$$

保证 $F_X(x)$ 是非减且连续。

概率的归一化条件

由于 $P(X \in (-\infty, +\infty)) = 1$ ，所以：

$$\int_{-\infty}^{+\infty} f_X(x) dx = 1$$

这是 PDF 必须满足的**归一化条件**。

5.1.4 离散 vs 连续：对比

概念	离散随机变量	连续随机变量
概率表示	质量函数 $P(X = x_i)$	密度函数 $f_X(x)$

单点概率	$P(X = x_i)$ 有意义且 > 0	$P(X = x) = 0$
计算概率	求和	积分
累积概率	累积概率 $F_X(x) = P(X \leq x)$	累积分布函数 $F_X(x) = \int_{-\infty}^x f_X(t)dt$

5.1.5 例子讲解

均匀分布 $U(a, b)$

- PDF:

$$f_X(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

- CDF:

$$F_X(x) = \begin{cases} 0 & x < a \\ \frac{x-a}{b-a} & a \leq x \leq b \\ 1 & x > b \end{cases}$$

验证:

$$\frac{d}{dx} F_X(x) = \frac{d}{dx} \frac{x-a}{b-a} = \frac{1}{b-a} = f_X(x)$$

标准正态分布 $N(0, 1)$

- PDF:

$$f_X(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

- CDF:

$$F_X(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-\frac{t^2}{2}} dt$$

无闭式表达，通常用数值积分或查表。

5.1.6 应用意义

- CDF** 常用于判定随机变量是否落在某区间的概率
- PDF** 适合用来分析变量的密度分布特征、估计概率密度、进行概率积分

5.1.7 几何意义

曲线下的面积 = 概率，PDF 是曲线的“高度”，CDF 是从左侧累计的面积。

这里，用Python实现高斯分布（正态分布）*的概率密度函数（PDF）和累积分布函数（CDF）。

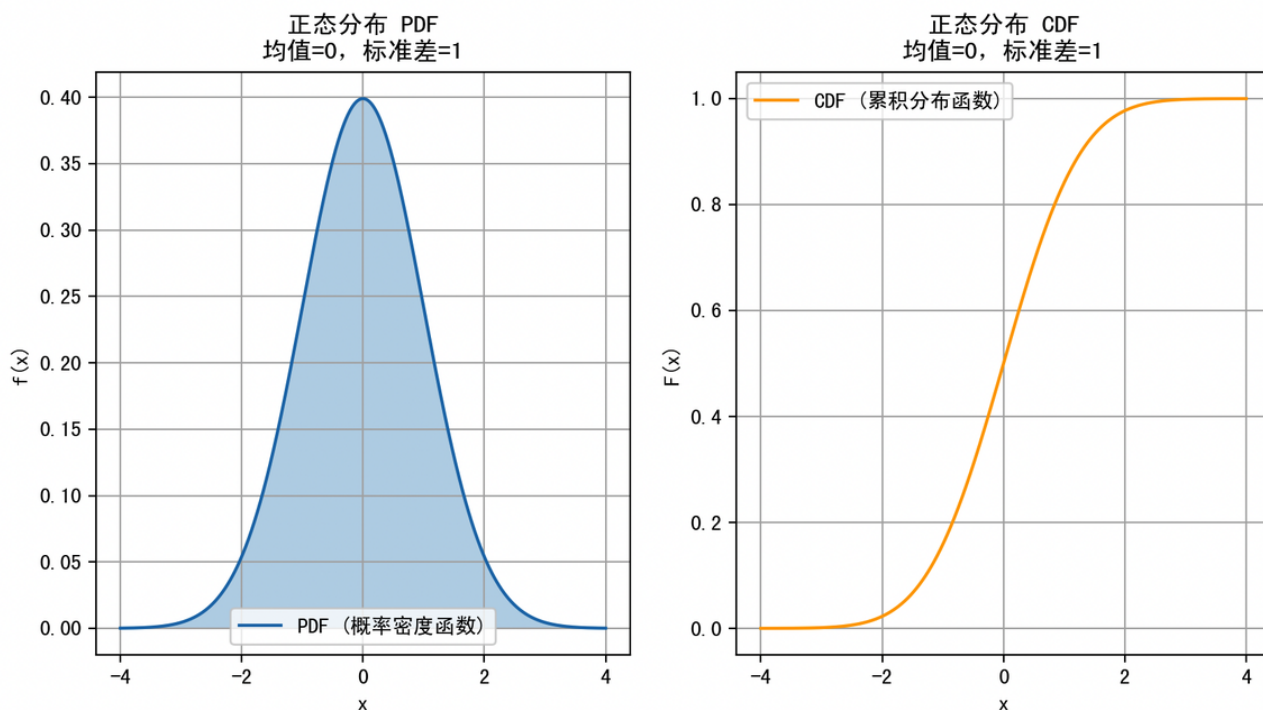
```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.stats import norm
4
5  # 设置高斯分布参数：均值mu，标准差sigma
6  mu = 0
7  sigma = 1
8
9  # 生成x轴数据，从-4到4，覆盖绝大部分概率质量
10 x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
11
12 # 计算对应点的PDF和CDF
13 pdf = norm.pdf(x, loc=mu, scale=sigma)
14 cdf = norm.cdf(x, loc=mu, scale=sigma)
15
16 # 绘制图形
17 plt.figure(figsize=(10,5))
18
19 # PDF
20 plt.subplot(1,2,1)
21 plt.plot(x, pdf, label='PDF (概率密度函数)')
22 plt.fill_between(x, pdf, alpha=0.3)
23 plt.title('正态分布 PDF\n均值=0, 标准差=1')
24 plt.xlabel('x')
25 plt.ylabel('f(x)')
26 plt.grid(True)
27 plt.legend()
28
29 # CDF
30 plt.subplot(1,2,2)
31 plt.plot(x, cdf, color='orange', label='CDF (累积分布函数)')
32 plt.title('正态分布 CDF\n均值=0, 标准差=1')
33 plt.xlabel('x')
34 plt.ylabel('F(x)')
35 plt.grid(True)
36 plt.legend()
37
38 plt.show()

```

其中：

- `norm.pdf(x, loc=mu, scale=sigma)`：计算点 `x` 对应的正态分布概率密度函数值。
- `norm.cdf(x, loc=mu, scale=sigma)`：计算点 `x` 对应的累积分布函数值，表示随机变量小于等于 `x` 的概率。



左边图是“钟形曲线”，表示概率密度。

右边图是“S”形曲线，表示累积概率。

数学上，PDF是概率分布的“密度”，积分（面积）才是概率。

CDF是从负无穷到 x 积分的结果，即：

$$F(x) = P(X \leq x) = \int_{-\infty}^x f(t)dt$$

5.1.8 小结

特点	PDF	CDF
定义	概率密度，概率的导数	累计概率
计算概率方法	积分区间	直接差值
取值范围	非负，积分为1	单调非减，取值在 $[0, 1]$
适用随机变量类型	连续	离散 & 连续皆适用

5.2 分布变换与生成过程

5.2.1 什么是分布变换？

在概率论中，**分布变换（Distribution Transformation）** 是指通过对随机变量进行某种函数映射，来得到新的随机变量及其分布的过程。

具体来说：

- 给定一个随机变量 X ，具有概率密度函数（PDF） $f_X(x)$ 或概率质量函数（PMF） $P_X(x)$ 。
- 通过某个函数 $Y = g(X)$ ，将 X 映射为新随机变量 Y 。
- 关键是要找到 Y 的概率分布 $f_Y(y)$ 或 $P_Y(y)$ 。

这个过程是概率论和统计学中非常重要的工具，常用于：

- 数据变换（如对数变换、标准化）
- 生成复杂分布的样本
- 解析复杂函数的概率性质

5.2.2 单变量连续随机变量的变换法则

设 X 是连续随机变量， $Y = g(X)$ 是 X 的单调可微函数。

目标：求 Y 的概率密度函数 $f_Y(y)$ 。

变换法则（公式推导）

1. 由随机变量定义， $Y = g(X)$ ，我们关注的是：

$$P(Y \leq y) = P(g(X) \leq y)$$

2. 如果 g 是单调递增函数，等价于：

$$P(X \leq g^{-1}(y))$$

3. 所以，累积分布函数 (CDF) $F_Y(y)$ 可写成：

$$F_Y(y) = P(Y \leq y) = P(X \leq g^{-1}(y)) = F_X(g^{-1}(y))$$

4. 对 y 求导（利用链式法则），得到概率密度函数：

$$f_Y(y) = \frac{d}{dy} F_Y(y) = f_X(g^{-1}(y)) \cdot \left| \frac{d}{dy} g^{-1}(y) \right|$$

这里的绝对值是因为要确保概率密度为非负。

例子：对数变换

- 假设 $X \sim \text{Exponential}(\lambda)$ ，即

$$f_X(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

- 定义 $Y = g(X) = \ln(X)$

求 $f_Y(y)$ ：

1. 先求 $g^{-1}(y) = e^y$

2. 计算导数：

$$\frac{d}{dy} g^{-1}(y) = \frac{d}{dy} e^y = e^y$$

3. 套用公式：

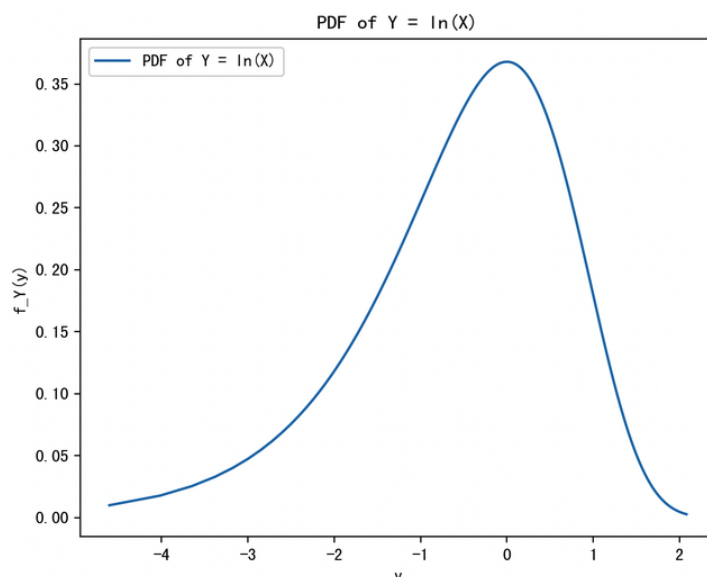
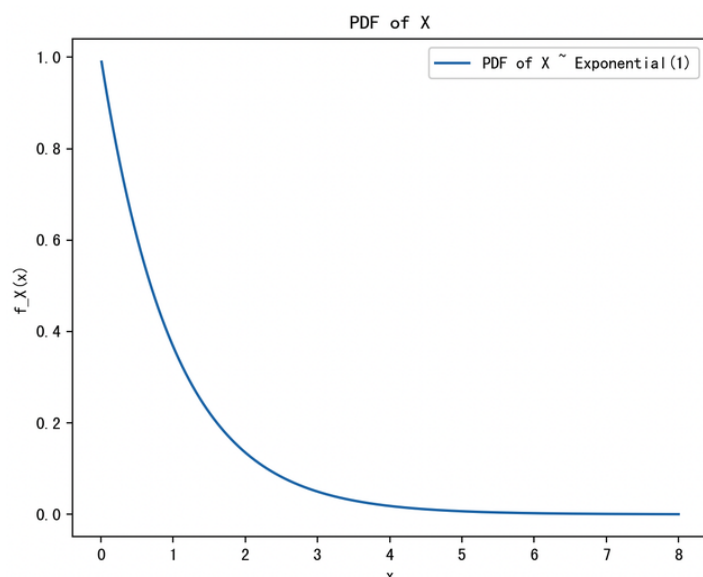
$$f_Y(y) = f_X(e^y) \cdot |e^y| = \lambda e^{-\lambda e^y} \cdot e^y, \quad y \in \mathbb{R}$$

下面，咱们用代码感受一下，根据变换公式计算新随机变量的概率密度：

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.stats import expon
4
5  # 生成指数分布样本
6  lambda_param = 1.0
7  x = np.linspace(0.01, 8, 1000) # x > 0
8  pdf_x = lambda_param * np.exp(-lambda_param * x)
9
10 # 对数变换 Y = ln(X)
11 y = np.log(x)
12 # 计算Y的pdf, f_Y(y) = f_X(e^y) * e^y
13 pdf_y = lambda_param * np.exp(-lambda_param * np.exp(y)) * np.exp(y)
14
15 plt.figure(figsize=(12,5))
16
17 plt.subplot(1,2,1)
18 plt.plot(x, pdf_x, label='PDF of X ~ Exponential(1)')
19 plt.title("PDF of X")
20 plt.xlabel("x")
21 plt.ylabel("f_X(x)")
22 plt.legend()
23
24 plt.subplot(1,2,2)
25 plt.plot(y, pdf_y, label='PDF of Y = ln(X)')
26 plt.title("PDF of Y = ln(X)")
27 plt.xlabel("y")
28 plt.ylabel("f_Y(y)")
29 plt.legend()
30
31 plt.tight_layout()
32 plt.show()

```



5.2.3 多变量随机变量的变换 (Jacobian变换)

对于多维连续随机变量：

$$\mathbf{X} = (X_1, X_2, \dots, X_n)$$

- 变换为 $\mathbf{Y} = (Y_1, Y_2, \dots, Y_n) = g(\mathbf{X})$

目标：找到 $\mathbf{Y}$ 的联合概率密度函数 $f_{\mathbf{Y}}(\mathbf{y})$

变换法则

- 如果变换是可逆且可微的，定义雅可比矩阵：

$$J = \frac{\partial(y_1, y_2, \dots, y_n)}{\partial(x_1, x_2, \dots, x_n)} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \dots & \frac{\partial y_n}{\partial x_n} \end{bmatrix}$$

- 逆变换 $\mathbf{X} = g^{-1}(\mathbf{Y})$ 存在。

- 概率密度函数满足：

$$f_{\mathbf{Y}}(\mathbf{y}) = f_{\mathbf{X}}(g^{-1}(\mathbf{y})) \cdot \left| \det \left(\frac{\partial g^{-1}(\mathbf{y})}{\partial \mathbf{y}} \right) \right|$$

或者等价地用正变换雅可比：

$$f_{\mathbf{Y}}(\mathbf{y}) = f_{\mathbf{X}}(g^{-1}(\mathbf{y})) \cdot \frac{1}{|\det(J)|}$$

其中 $\det(J)$ 是雅可比矩阵的行列式。

例子：二维极坐标变换

- 定义二维变量 $\mathbf{X} = (X_1, X_2)$ (笛卡尔坐标)
- 变换为极坐标 $\mathbf{Y} = (R, \Theta)$,

$$R = \sqrt{X_1^2 + X_2^2}, \quad \Theta = \arctan \left(\frac{X_2}{X_1} \right)$$

假设 $\mathbf{X}$ 的联合密度为 $f_{\mathbf{X}}(x_1, x_2)$ ，要找 $f_{\mathbf{Y}}(r, \theta)$ 。

1. 求逆变换：

$$X_1 = r \cos \theta, \quad X_2 = r \sin \theta$$

2. 计算雅可比行列式：

$$J = \begin{bmatrix} \frac{\partial X_1}{\partial r} & \frac{\partial X_1}{\partial \theta} \\ \frac{\partial X_2}{\partial r} & \frac{\partial X_2}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{bmatrix}$$

3. 计算行列式：

4. 这里的变换是从 (r, θ) 到 (x, y) 的正向变换，因此雅可比应写为 $\frac{\partial(x, y)}{\partial(r, \theta)}$ ，行列式为 r 。

$$\det(J) = \begin{vmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{vmatrix} = r \cos^2 \theta + r \sin^2 \theta = r(\cos^2 \theta + \sin^2 \theta) = r$$

4. 代入变换公式：

$$f_{R, \Theta}(r, \theta) = f_{X_1, X_2}(r \cos \theta, r \sin \theta) \cdot r$$

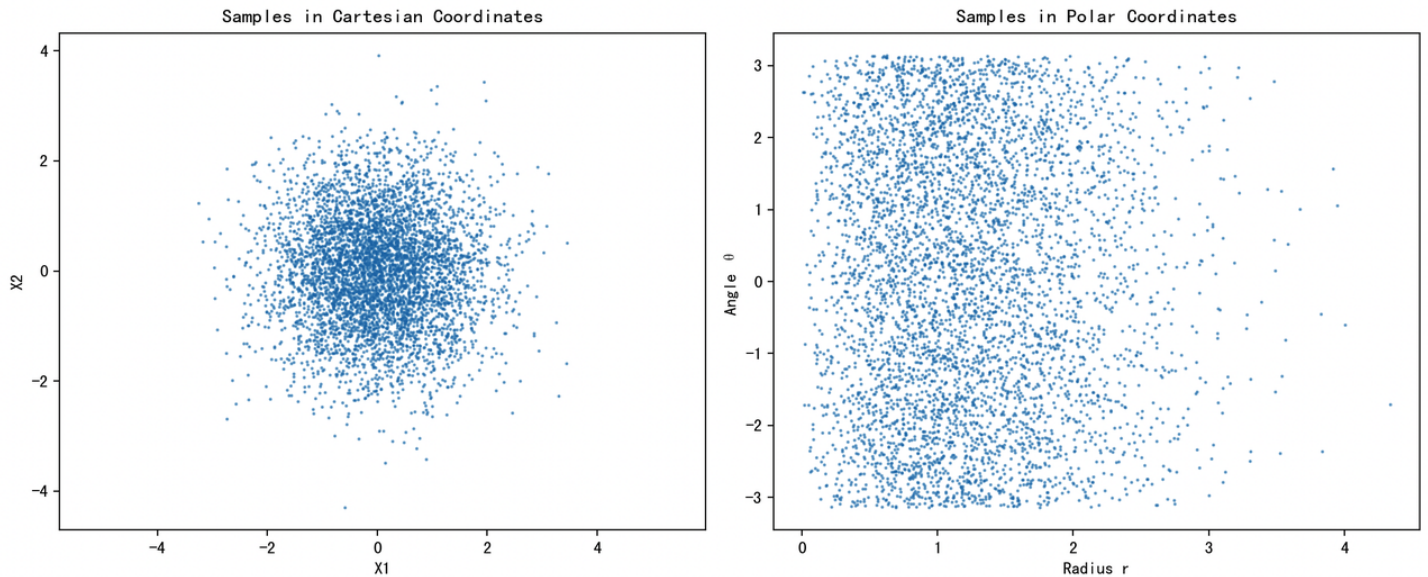
代码实现多维变量变换的几何效果（通过极坐标映射）：

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # 生成二维正态分布样本
5  mean = [0, 0]
6  cov = [[1, 0], [0, 1]] # 独立标准正态
7  samples = np.random.multivariate_normal(mean, cov, size=5000)
8
9  x1 = samples[:,0]
10 x2 = samples[:,1]
11
12 # 极坐标转换
13 r = np.sqrt(x1**2 + x2**2)
14 theta = np.arctan2(x2, x1)
15
16 plt.figure(figsize=(12,5))
17
18 plt.subplot(1,2,1)
19 plt.scatter(x1, x2, s=1, alpha=0.5)
20 plt.title("Samples in Cartesian Coordinates")
21 plt.xlabel("X1")
22 plt.ylabel("X2")
23 plt.axis('equal')
24
25 plt.subplot(1,2,2)
26 plt.scatter(r, theta, s=1, alpha=0.5)
```

```

27 plt.title("Samples in Polar Coordinates")
28 plt.xlabel("Radius r")
29 plt.ylabel("Angle θ")
30
31 plt.tight_layout()
32 plt.show()

```



5.2.4 生成过程（随机变量采样）

生成过程 是指通过已知分布或者通过分布变换来产生符合特定概率分布的随机样本。

逆变换采样法

适用于一维连续随机变量：

- 给定随机变量 X 的CDF $F_X(x)$,
- 令 $U \sim \text{Uniform}(0, 1)$ 是均匀分布随机变量,
- 定义：

$$X = F_X^{-1}(U)$$

则 X 的分布即为所需分布。

例子：从指数分布采样

指数分布的CDF为：

$$F_X(x) = 1 - e^{-\lambda x}$$

求逆CDF：

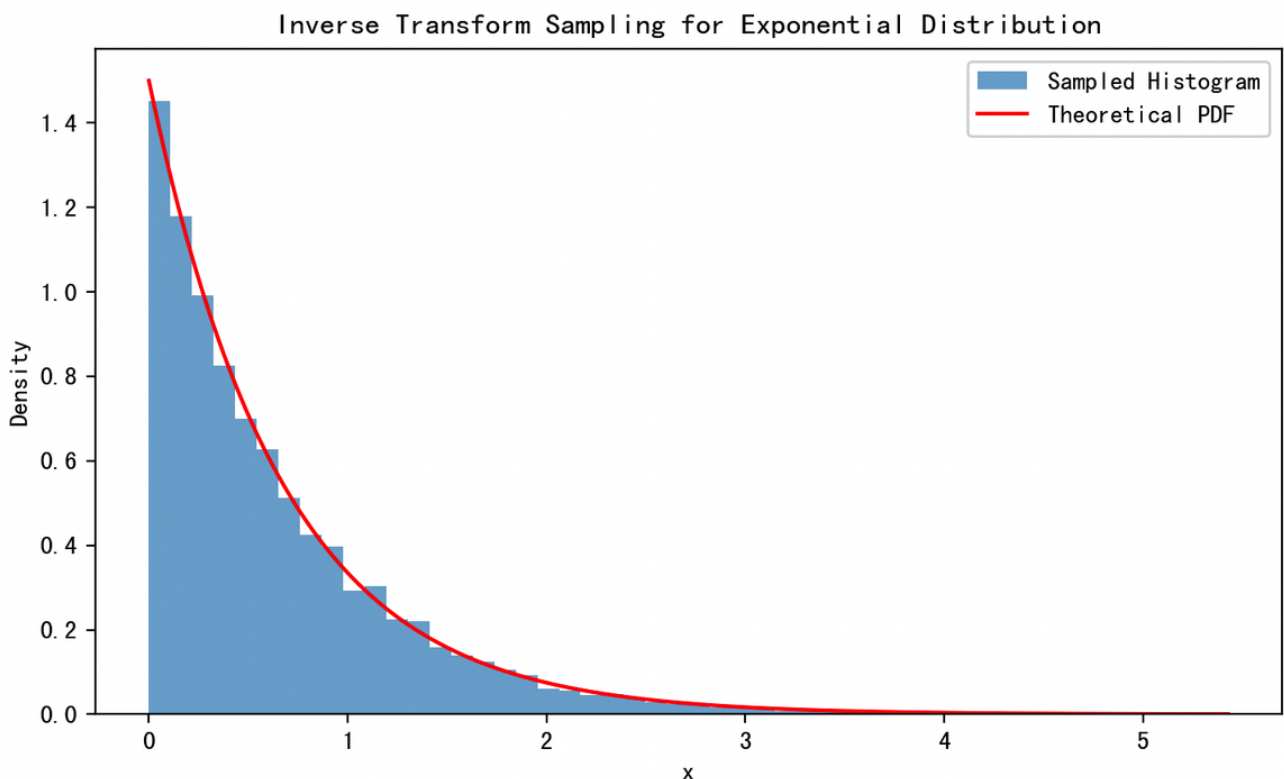
$$F_X^{-1}(u) = -\frac{1}{\lambda} \ln(1 - u)$$

令 $U \sim \text{Uniform}(0, 1)$, 则：

$$X = -\frac{1}{\lambda} \ln(1 - U)$$

服从指数分布。

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  lambda_param = 1.5
5  N = 10000
6
7  # 生成均匀样本
8  U = np.random.uniform(0, 1, N)
9
10 # 利用逆CDF采样公式
11 X = -np.log(1 - U) / lambda_param
12
13 # 画出样本的直方图与理论PDF
14 x_vals = np.linspace(0, np.max(X), 1000)
15 pdf_vals = lambda_param * np.exp(-lambda_param * x_vals)
16
17 plt.hist(X, bins=50, density=True, alpha=0.6, label='Sampled Histogram')
18 plt.plot(x_vals, pdf_vals, 'r-', label='Theoretical PDF')
19 plt.title("Inverse Transform Sampling for Exponential Distribution")
20 plt.xlabel("x")
21 plt.ylabel("Density")
22 plt.legend()
23 plt.show()
```



多变量采样：对于复杂多变量分布，直接采样较难，可借助分布变换、马尔科夫链蒙特卡洛（MCMC）、变分推断等技术，变换与生成过程密切相关，是概率建模核心。

5.2.5 总结与关键点

关键点	说明
单变量变换	利用反函数及其导数，调整概率密度
多变量变换	利用雅可比行列式调整联合概率密度
逆变换采样法	用均匀分布采样通过CDF反函数产生目标分布随机变量
生成过程核心	通过变换和采样产生符合给定分布的随机样本
应用	数据标准化、模拟实验、生成模型、强化学习等

5.3 应用：采样、变量标准化、数据增强

5.3.1 采样 Sampling

背景：
在机器学习和统计推断中，采样是构建数据分布模型、估计参数、训练神经网络（如GAN）等基础。

理论基础：
假设我们希望从一个复杂分布 $p(x)$ 中采样。但很多时候我们无法直接从 $p(x)$ 采样。于是用**变换方法**构造采样器。

方法一：逆变换采样法（Inverse Transform Sampling）

如果 $X \sim p(x)$ ，累积分布函数为 $F(x)$ ，则：

$$U \sim \text{Uniform}(0, 1)$$

$X = F^{-1}(U)$ 即为所需样本

适用于分布具有封闭形式的 F^{-1} （如指数分布、幂律分布）。

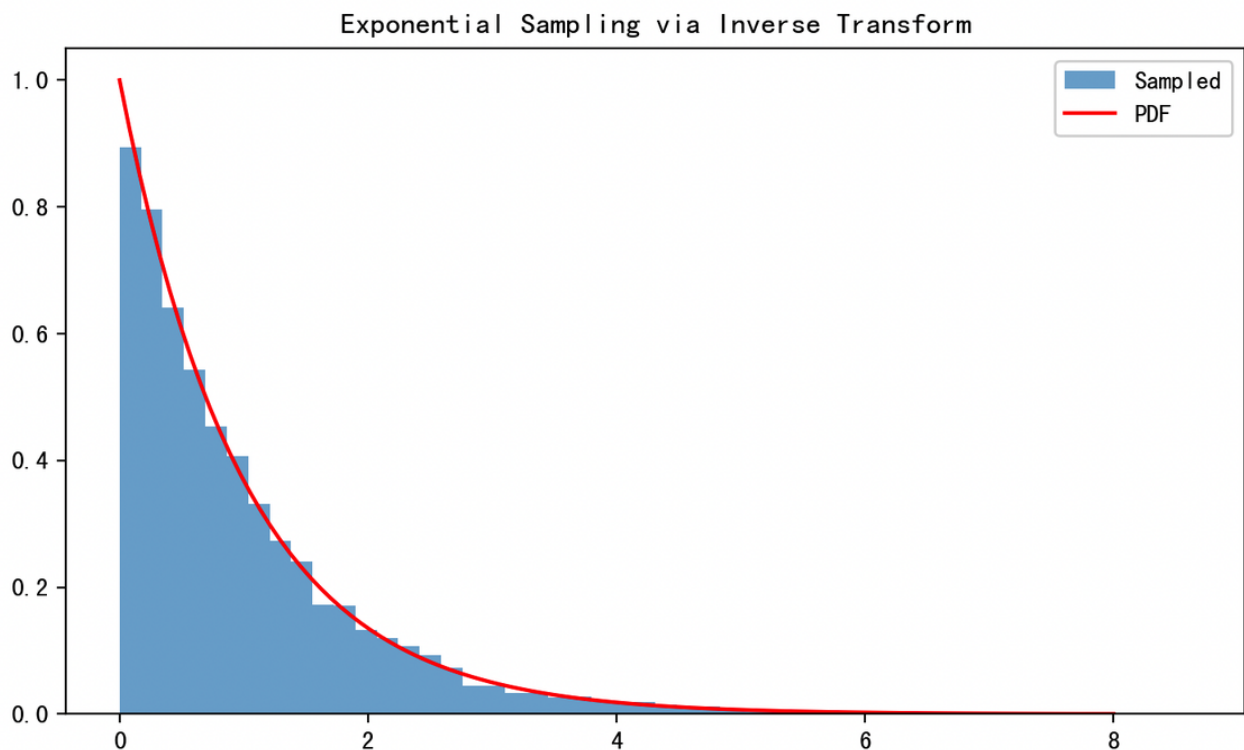
示例：从指数分布 $f(x) = \lambda e^{-\lambda x}, x \geq 0$ 中采样

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  lambda_param = 1.0
5  n = 10000
6  U = np.random.uniform(0, 1, size=n)
7  X = -np.log(1 - U) / lambda_param
8
```

```

9 plt.hist(X, bins=50, density=True, alpha=0.6, label='Sampled')
10 x_vals = np.linspace(0, 8, 100)
11 plt.plot(x_vals, lambda_param * np.exp(-lambda_param * x_vals), 'r-',
12         label='PDF')
13 plt.title("Exponential Sampling via Inverse Transform")
14 plt.legend()
15 plt.show()

```



方法二：重参数技巧（Reparameterization Trick）

目标：从可微分的**确定性函数**中生成复杂分布的样本。

例：若 $Z \sim \mathcal{N}(0, 1)$ ，则 $X = \mu + \sigma Z \sim \mathcal{N}(\mu, \sigma^2)$

在深度学习中非常关键，VAEs 等模型依赖它进行梯度传播。

```

1 mu, sigma = 2.0, 0.5
2 z = np.random.normal(0, 1, 1000)
3 x = mu + sigma * z # 从 N(mu, sigma^2) 中采样

```

5.3.2 变量标准化

背景：

不同特征的取值尺度不同会干扰模型训练。通过标准化可以加速收敛、提升精度。

理论原理：

常见标准化方法：

- **Z-score 标准化：**

$$x' = \frac{x - \mu}{\sigma}$$

- 使得 $x' \sim \mathcal{N}(0, 1)$ （标准正态）

- **Min-Max 归一化：**

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- 将变量映射到 $[0, 1]$

为什么标准化等价于分布变换？

因为它将原分布通过线性函数映射到另一个具有统一尺度的分布。例如：

$$x \sim \mathcal{N}(\mu, \sigma^2) \Rightarrow x' = \frac{x - \mu}{\sigma} \sim \mathcal{N}(0, 1)$$

这是一种“密度保留”的仿射变换。

Python 示例：

```
1  from sklearn.preprocessing import StandardScaler, MinMaxScaler
2  import numpy as np
3
4  X = np.random.normal(loc=10, scale=3, size=(100, 1))
5
6  # Z-score标准化
7  scaler_std = StandardScaler()
8  X_std = scaler_std.fit_transform(X)
9
10 # Min-Max归一化
11 scaler_minmax = MinMaxScaler()
12 X_minmax = scaler_minmax.fit_transform(X)
13
14 print("原始均值：", X.mean(), "标准差：", X.std())
15 print("Z-score 后：", X_std.mean(), "标准差：", X_std.std())
```

5.3.3 数据增强

背景：

在深度学习中，数据增强通过对训练样本进行**平滑变换**或**采样扰动**来扩展数据集，增强模型泛化能力。

概率变换的视角：

数据增强的每一个操作都可以视为从某种**变换概率分布**中采样后进行映射：

- 图像平移：仿射变换
- 噪声扰动：添加高斯噪声 $\epsilon \sim \mathcal{N}(0, \sigma^2)$
- Dropout：从 Bernoulli 分布中采样 “保留掩码”

数学建模举例：

1. 添加高斯噪声（对数变换）

$$x' = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

2. Dropout

$$x' = x \cdot m, \quad m_i \sim \text{Bernoulli}(p)$$

Python 示例：向特征添加高斯噪声

```
1  def add_noise(x, sigma=0.1):
2      noise = np.random.normal(0, sigma, size=x.shape)
3      return x + noise
4
5  x = np.linspace(-1, 1, 100)
6  x_noisy = add_noise(x, sigma=0.2)
7
8  plt.plot(x, label="Original")
9  plt.plot(x_noisy, label="Noisy")
10 plt.legend()
11 plt.title("Additive Gaussian Noise for Data Augmentation")
12 plt.show()
```

5.3.4 小结

应用	数学基础	典型方法	工程作用
采样	密度变换、逆CDF	Inverse Sampling, Reparam Trick	分布学习、生成建模
变量标准化	仿射变换	Z-Score、Min-Max	提升模型训练稳定性
数据增强	噪声扰动、函数变换	加噪、Dropout、仿射变换	防止过拟合、提高泛化能力

这些操作都可以视为在原始数据分布基础上进行密度的变换与采样操作，从而达到建模、推断或增强的目标。